

Algoritmos de ordenamiento

Capítulo 4 — Análisis de Algoritmos con Julia

Eric S. Téllez

Tabla de contenidos

1	Algoritmos de ordenamiento	1
1.1	Introducción	1
1.1.1	Costo del problema	2
1.2	Algoritmos de ordenamiento	3
1.2.1	Bubble sort	3
1.2.2	Insertion sort	4
1.2.3	Heap sort	6
1.2.3.1	Ejercicios	7
1.2.4	Merge sort	7
1.2.4.1	Ejercicios	8
1.2.5	Quick sort	8
1.2.5.1	Ejercicios	10
1.3	Skip list	10
1.3.1	Ejercicios:	11
1.4	Lecturas	11
1.5	Material audio-visual sobre algoritmos de ordenamiento	12
1.6	Actividades	12
1.6.1	Actividad 1: Explotando la localidad y tamaños de instancia	12
1.6.2	Entregable	13

1. Algoritmos de ordenamiento

1.1. Introducción

En este tema se aborda el ordenamiento basado en comparación, esto es, existe un operador $<$ que es capaz de distinguir si un elemento a es menor que un elemento b .

El operador cumple con las siguientes propiedades:

- si $a < b$ y $b < c$ entonces $a < c$ (transitividad); e.g., $1 < 10$ y $10 < 100$ entonces $1 < 100$.
- tricotomía:

- si $a < b$ es falso y $b < a$ es falso, entonces $a = b$ (antisimetría); dicho de otras formas:
 - si a no es menor que b ni b menor que a entonces a es igual a b ,
 - sustituyendo variables, $1 < 1$ es falso, el intercambio es obvio, entonces $1 = 1$.
- en otro caso, $a < b$ o $b < a$.

1

Sin pérdida de generalidad, podemos plantear el problema de ordenamiento sin permitir repeticiones como sigue: dado un arreglo $A[1, n] = a_1, a_2, \dots, a_n$; un algoritmo de ordenamiento obtiene la permutación π tal que $a_{\pi(1)} < a_{\pi(2)} < \dots < a_{\pi(n)}$.

2

En términos prácticos, la idea es reorganizar A , mediante el cálculo implícito de la permutación π , de tal forma que después de terminar el proceso de ordenamiento se obtenga que A está ordenado, i.e., $a_i \leq a_{i+1}$. En sistemas reales, el alojar memoria para realizar el ordenamiento implica costos adicionales, y es por esto que muchas veces se busca modificar directamente A .

3

1.1.1. Costo del problema

Para una entrada de tamaño n existen $n!$ permutaciones posibles; cada una de estas permutaciones es una instancia del problema de ordenamiento de tamaño n .

Existe una permutación objetivo π^* , i.e., que cumple con la definición de que está ordenada; ahora pensemos en un grafo donde cada π_i está conectada con todas las permutaciones en las que se puede transformar haciendo una única operación, e.g., intercambiando un elemento. El algoritmo forma ese grafo con sus posibles decisiones, por lo que el camino más largo i.e., *ruta sin ciclos*, entre cualquier π_i y la permutación π^* es el costo de peor caso del algoritmo.

Ahora, cada operación que realicemos en un algoritmo nos acercará más a π^* , descartando una cierta cantidad de instancias posibles pero no viables; si nuestra función de transición en el grafo viene dada con respecto a colocar cada par de elementos en su orden relativo, entonces, la mitad de las permutaciones se han descartado, ya que ese par no puede estar en el orden contrario. Por tanto, el costo de cualquier algoritmo que realice comparaciones y descarte la mitad del espacio de búsqueda, es $\log_2(n!)$, que usando la aproximación de Stirling,⁴ lo podemos reescribir como sigue:

¹Usar un operador como $<$ es suficiente para crear algoritmos correctos y eficientes, sin embargo, en la práctica y en una computadora real, también es válido utilizar operadores como $=$ o $\leq$, o intercambiar por $>$ y $\geq$ según convenga. No hay impacto en la eficiencia.

²Cuando se permiten elementos repetidos, se le llama *ordenamiento estable* y se asegura que en el arreglo ordenado se preserve el orden posicional original cuando $a = b$. Esta propiedad es importante cuando hay datos satelitales asociados a la llave de comparación.

³Utilizar π solo es necesario cuando no es posible modificar A . También es muy común utilizar datos *satélite* asociados con los valores a comparar, de esta manera es posible ordenar diversos tipos de datos. Un ejemplo de esto es ordenar un *dataframe*, pero también estructuras de datos donde existe un campo especial y el resto de los datos asociados es de importancia para una aplicación.

⁴Aproximación de Stirling https://en.wikipedia.org/wiki/Stirling%27s_approximation.

$$\log_2(n!) = n \log_2 n - n \log_2 e + O(\log_2 n)$$

Esto se puede simplemente escribir como $O(n \log n)$.

1.2. Algoritmos de ordenamiento

Existen muchos algoritmos que pueden resolver el problema de ordenamiento, es común contar el número de comparaciones ya que produce la información necesaria para la navegación en el grafo de instancias; también es común contar las operaciones de intercambio de elementos. Las pruebas y la navegación en el grafo determinan el costo del algoritmo. Es necesario mencionar que mover datos entre diferentes zonas de memoria puede llegar a ser más costoso que solo acceder a esas zonas, por lo que hay una asimetría en el costo de estas dos operaciones.

Note que algunos de los algoritmos más simples pueden tener un comportamiento oportunista y que son capaces de obtener ventaja en instancias sencillas, por lo que no debería saltarse esas secciones si solo conoce su comportamiento en peor caso.

1.2.1. Bubble sort

El algoritmo de ordenamiento de burbuja o *bubble sort* realiza una gran cantidad de comparaciones, como puede verse en Listado 1, el algoritmo usa dos ciclos anidados para realizar una comparación y una posible transposición, formando un *triángulo*, i.e.,

$$\sum_{i=1}^{n-1} \sum_{j=1}^{n-i} O(1);$$

por lo tanto su costo está dominado por el triángulo formado, i.e., $\sim n^2/2$ lo que puede escribirse simplemente como $O(n^2)$.

- ① Ciclo que recorre $n - 1$ veces todo el arreglo; y pone el elemento máximo en su posición final.
- ② Ciclo que recorre $n - i$ veces el arreglo; ya que en cada corrida se pone el máximo en su posición.
- ③ Intercambio cuando hay pares en desorden.

8-element Vector{Int64}:

```
1
2
3
4
5
6
7
8
```

Listado 1 Bubble sort de peor caso

```
function bubble_sort!(A)
    n = length(A)
    for i in 1:n-1
        for j in 1:n-i
            if A[j] > A[j+1]
                A[j], A[j+1] = A[j+1], A[j]
            end
        end
    end

    A
end

bubble_sort!([8, 4, 3, 1, 6, 5, 2, 7])
```

El algoritmo mostrado en Listado 1 es un algoritmo de peor caso, ya que sin importar la complejidad de la instancia (i.e., qué tan alejada está π_i de π^*), se comporta igual.

Es relativamente fácil hacer un bubble sort que tenga en cuenta la complejidad de la instancia, medida como el número de intercambios necesarios.

- ① La idea es que si no hay intercambios en una iteración, entonces el arreglo ya está ordenado.
- ② Contador de intercambios.
- ③ Condición de paro, i.e., no hubo intercambios.

8-element Vector{Int64}:

1
2
3
4
5
6
7
8

En la forma Listado 2, bubble sort es capaz de terminar en $n - 1$ comparaciones si el arreglo está ordenado, sacando provecho de casos simples en términos de instancias casi ordenadas.

1.2.2. Insertion sort

El algoritmo de ordenamiento por inserción o *insertion sort* es un algoritmo simple que al igual que bubble sort tiene un mal peor caso y puede aprovechar casos simples.

Listado 2 Bubble sort adaptable

```
function adaptive_bubble_sort!(A)
    n = length(A)

    for i in 1:n-1
        s = 0
        for j in 1:n-i
            if A[j] > A[j+1]
                s += 1
                A[j], A[j+1] = A[j+1], A[j]
            end
        end
        s == 0 && break
    end

    A
end

adaptive_bubble_sort!([7, 8, 4, 3, 1, 6, 5, 2])
```

- ① El algoritmo comienza en la segunda posición del arreglo y revisará todos los elementos.
- ② Es importante hacer una copia de *key* para simplificar la implementación.
- ③ La idea general es ordenar las posiciones de $1..i$, para esto se debe recorrer hacia atrás el arreglo completo, para determinar la posición de inserción de *key*.
- ④ Intercambio de elementos para colocar *key* en su lugar ordenado.
- ⑤ *key* se pone en su lugar final.

8-element Vector{Int64}:

1
2
3
4
5
6
7
8

Para analizar Listado 3, es importante notar que el ciclo más externo termina con el subarreglo $A[1..i]$ ordenado; por lo que cuando se comienza el ciclo, si *key* se prueba estar en su posición correcta, entonces ya no es necesario revisar el resto del subarreglo, esto determina que un arreglo ordenado tendrá un costo de $O(n)$ comparaciones; si está *casi ordenado* en términos del número de intercambios necesarios, entonces, el algoritmo se adaptará sacando provecho de la instancia.

Listado 3 Algoritmo *insertion sort*

```
function insertion_sort!(A)
    n = length(A)
    for i in 2:n
        key = A[i]
        j = i - 1
        while j >= 1 && A[j] > key
            A[j + 1] = A[j]
            j -= 1
        end

        A[j + 1] = key
    end

    A
end

insertion_sort!([5, 1, 4, 8, 2, 6, 3, 7])
```

En el *peor caso* de insertion sort, el algoritmo no puede parar de manera prematura, e.g., un arreglo en orden reverso, el ciclo **for** se ejecutará $n - 1$ veces, mientras que el ciclo **while** deberá revisar el subarreglo completo en cada iteración, sumando un costo de i operaciones en cada iteración, i.e., $\sum_{i=1}^n i$, esta forma produce un *triángulo*, resultando en un costo $O(n^2)$.

1.2.3. Heap sort

El algoritmo de ordenamiento por *heap* o *heap sort* utiliza una estructura de datos implícita llamada *heap* binario (*binary heap*). Un *heap* binario puede visualizarse como un árbol binario casi completo mapeado directamente sobre las posiciones de un arreglo continuo. Para ser considerado un *heap*, el arreglo debe mantener una propiedad estructural jerárquica conocida como la **propiedad del *heap***: el valor de cada nodo padre siempre debe ser mayor o igual al de sus hijos (conocido como *max-heap*) o menor o igual al de sus hijos (*min-heap*).

El ejemplo a continuación construye un ***max-heap***. La decisión de usar un *max-heap* se debe a que la estrategia del algoritmo para ordenar el arreglo en forma ascendente consiste en asegurar sistemáticamente que el elemento de mayor valor quede en la raíz (la posición 1 del arreglo). Una vez obtenido el máximo, éste se intercambia con el último elemento disponible en la parte desordenada del arreglo y se aísla, reduciendo el tamaño del problema de manera iterativa.

A diferencia de *insertion sort* o *bubble sort*, su costo en el peor de los casos es de $O(n \log n)$, haciéndolo asintóticamente óptimo para ordenamiento por comparación.

- ① Los hijos izquierdo y derecho de un nodo i en un arreglo indexado desde 1.
- ② Intercambio de elementos para cumplir la propiedad de *max-heap*.

- ③ Llamada recursiva para propagar el cambio hacia abajo.
- ④ Fase de construcción del *heap* (*build heap*); los nodos hoja ya cumplen la propiedad.
- ⑤ Fase de extracción del máximo, colocándolo al final del arreglo e ignorándolo en el tamaño restante.

8-element Vector{Int64}:

1
2
3
4
5
6
7
8

A pesar de su cota $O(n \log n)$, *heap sort* suele ser más lento en la práctica que *quick sort* o *merge sort* debido a su baja localidad de referencia y al número de operaciones adicionales para mantener el *heap*.

1.2.3.1. Ejercicios

- 1. ¿Cuáles son las ventajas y desventajas de *heap sort*?
- 2. Modifique la implementación para que ordene en orden descendente utilizando un *min-heap*.

1.2.4. Merge sort

Merge sort es el algoritmo clásico del paradigma de *dividir para vencer*. Divide el arreglo en mitades, ordena recursivamente cada una y luego fusiona (*merge*) los dos subarreglos resultantes. Al igual que *heap sort*, tiene un costo de $O(n \log n)$ garantizado.

- ① Copiamos los elementos a un arreglo temporal en el mismo rango.
- ② Comparamos los frentes de ambas mitades ordenadas y los escribimos en su posición original.
- ③ Vaciamos los elementos restantes de la mitad izquierda (la derecha ya está en su posición final por omisión).
- ④ El arreglo temporal **temp** es pre-asignado de tamaño idéntico a **A** para evitar múltiples reservas de memoria.
- ⑤ Fusión de las partes en un tiempo lineal al tamaño del rango.

8-element Vector{Int64}:

1
2
3
4
5

6
7
8

La gran ventaja de *merge sort* es que, a diferencia de *quick sort*, su costo en peor caso es siempre $O(n \log n)$ y es un algoritmo de ordenamiento estable. El costo principal radica en la memoria extra necesaria ($O(n)$) y en las copias adicionales durante la fusión.

1.2.4.1. Ejercicios

1. ¿Cuáles son las ventajas y desventajas de *merge sort*?
2. ¿Por qué *merge sort* se puede utilizar eficientemente en algoritmos paralelos mientras que otros pueden tener muchas dificultades?
3. ¿Cómo se puede reducir la memoria extra necesaria de *merge sort*?

1.2.5. Quick sort

Quick sort (ver Cormen et al. 2022, cap. 7) es un algoritmo tipo *dividir para vencer*; esto es, un algoritmo que divide un problema grande en instancias pequeñas más sencillas. Es uno de los algoritmos más veloces en la práctica por su buen manejo de memoria, aun cuando tiene un peor caso cuadrático, en promedio el costo es $O(n \log n)$.

- ① El arreglo se divide en 3 partes, ordenadas entre sí, un subarreglo izquierdo, un pivote, y un subarreglo derecho; los subarreglos no están ordenados localmente, pero el pivote está en su posición final.
- ② Se resuelve el problema izquierdo y el problema derecho por separado.
- ③ La función *part!* particiona el arreglo $A[low : end]$ en 3 partes como se especificó en el punto 1; para eso selecciona de manera aleatoria un pivote. Lo ponemos al final del arreglo para simplificar el código siguiente.
- ④ Este ciclo itera por todo el subarreglo, su objetivo es asegurar que $A[i] < piv$ para todo $i \in low : piv - 1$ y $piv < A[i]$ para todo $i \in piv + 1 : high$.
- ⑤ Intercambia elementos si $A[j] < piv$, hacemos seguimiento de i ya que esta posición determinará al pivote.
- ⑥ Como *piv* se encontraba en *high*, entonces hay que intercambiarlos para que *qsort!* sepa como manejarlos; recordando que los subarreglos no están ordenados dentro de sí.

8-element Vector{Int64}:

1
2
3
4
5
6
7
8

El código Listado 6 es relativamente simple, usa recurrencias sobre *qsort!* sobre dos partes extremas divididas por un pivote; estos tres elementos son encontrados en *part!*. La función *part!* es muy eficiente en términos de memoria, lo que puede hacer la diferencia en la práctica. La correcta selección del pivote es muy importante para evitar casos malos, i.e., costo cuadrático; en esta implementación se realiza una selección aleatoria de pivote que funcionará en la mayoría de los casos.

El peor de los casos en *qsort!* es debido a una mala selección del pivote, de tal forma que

$$|A[low : piv - 1]| \ll |A[piv + 1 : high]|,$$

o lo contrario en toda selección, en el extremo uno de los subarreglos puede verse como de tamaño constante o cero, i.e., selección de pivote como el *mínimo* o el *máximo*. Esta estrategia reduce a *qsort!* a un costo $O(n^2)$.

Si se realiza un particionado perfecto donde

$$|A[low : piv - 1]| \approx |A[piv + 1 : high]|,$$

tenemos un comportamiento óptimo; la recursión se divide en dos partes casi iguales y la profundidad máxima del árbol de llamadas será $\log_2 n$. Como en cada nivel la función *part!* realiza un esfuerzo lineal $O(n)$, el costo total de la ejecución es $O(n \log n)$.

Para comprender por qué el **caso promedio** también es de $O(n \log n)$, considere que no es estrictamente necesario un particionado balanceado para lograr alta eficiencia. Incluso si la partición resultara constantemente desbalanceada de manera proporcional, por ejemplo, dividiendo siempre el arreglo en una fracción de $1/10$ y $9/10$, la altura del árbol de recursividad sería $\log_{10/9} n$, lo cual sigue siendo asintóticamente $O(\log n)$. En cualquier nivel del árbol, el costo de particionar las sublistas no superará a $O(n)$, por lo que el trabajo total en todo el árbol seguirá acotado por $O(n \log n)$.

En términos más formales, dado que nuestra implementación elige el pivote de forma aleatoria, cualquier índice para la posición final del pivote tiene la misma probabilidad $1/n$. La ecuación de recurrencia que define el costo esperado $\mathbb{E}[T(n)]$ es el promedio sobre las n posibles divisiones:

$$\mathbb{E}[T(n)] = \Theta(n) + \frac{1}{n} \sum_{k=0}^{n-1} (\mathbb{E}[T(k)] + \mathbb{E}[T(n-k-1)])$$

La solución matemática de esta sumatoria demuestra que $\mathbb{E}[T(n)] = \Theta(n \log n)$.



Intuición para resolver la recurrencia

Resolver la ecuación anterior puede parecer intimidante debido a la sumatoria, pero se basa en simplificaciones y trucos algebraicos clásicos:

1. **Simetría:** Note que $\mathbb{E}[T(k)]$ y $\mathbb{E}[T(n-k-1)]$ generan exactamente los mismos sumandos en direcciones opuestas. La ecuación se reduce a $\mathbb{E}[T(n)] = \Theta(n) + \frac{2}{n} \sum_{k=0}^{n-1} \mathbb{E}[T(k)]$.
2. **Eliminación de la fracción:** Multiplicando por n ambos lados, obtenemos $n\mathbb{E}[T(n)] = n\Theta(n) + 2 \sum_{k=0}^{n-1} \mathbb{E}[T(k)]$.
3. **Restado telescópico:** Para eliminar la sumatoria, podemos construir la misma ecuación para $n-1$, es decir $(n-1)\mathbb{E}[T(n-1)]$, y luego restarla a la ecuación de n . Al hacerlo,

casi toda la sumatoria se cancela dejando un solo término $\mathbb{E}[T(n-1)]$.

4. **Números Armónicos:** Al agrupar y dividir la ecuación resultante entre $n(n+1)$, la recurrencia toma la forma recurrente iterativa $\frac{\mathbb{E}[T(n)]}{n+1} = \frac{\mathbb{E}[T(n-1)]}{n} + \Theta\left(\frac{1}{n}\right)$. La expansión natural de estos incrementos corresponde a la serie armónica $H_n = \sum_{i=1}^n \frac{1}{i}$, cuyo crecimiento asintótico es bien sabido que está acotado por $\ln n$. Al multiplicar de vuelta por $n+1$, surge el factor $\Theta(n \log n)$.

Además de esta excelente cota teórica probabilística, las operaciones de *quick sort* ocurren completamente en su propio espacio (*in-place*) con bucles muy pequeños. Esto beneficia tremendamente la localidad de referencia en el caché del procesador, explicando por qué suele vencer en velocidad a algoritmos con la misma cota como *merge sort* o *heap sort* en pruebas reales de laboratorio.

1.2.5.1. Ejercicios

1. Evalúe la implementación de *quick sort* y compárela con *bubble sort adaptativo*.
2. Modifique la función *part!* para elegir el pivote utilizando el método de la “mediana de tres” (elija tres elementos al azar y use su mediana como pivote).

1.3. Skip list

Una *skip list* (Pugh 1990) es una lista ligada con capacidades de búsqueda eficiente con garantías probabilísticas, esto es que se cumplen con alta probabilidad. Para esto, la idea es que cada dato tiene asociado un arreglo de punteros o referencias hacia nodos sucesores, i.e., los nodos a nivel i se conectan con el siguiente nodo a nivel i . En el nivel más bajo, la *skip list* es una simple lista ligada, mientras que sube, se vuelve más dispersa dando saltos más largos.

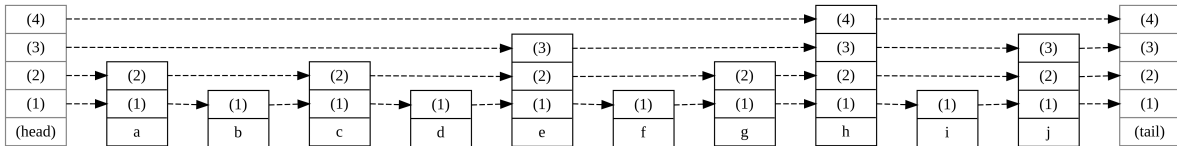


Figura 1: Ejemplo de una skip list

A diferencia de los algoritmos vistos anteriormente, en este caso, ya se tiene una estructura de datos, que conlleva un costo en memoria explícito por nodo. Figura 1 ilustra la estructura.

La altura de cada nodo es calculada de manera probabilística, dada la probabilidad p . Un valor común de $p = 0.5$. La altura de cada nodo se calcula como sigue:

```
function levels(p)
  i = 1
  while rand() < p
    i += 1
```

```

end

i
end

levels (generic function with 1 method)

```

Si tenemos n evaluaciones de *levels*, los niveles pequeños son relativamente probables, mientras que niveles grandes son relativamente poco probables. De hecho, los niveles $\log_{1/p} n$ son cercanos a una constante, $\log_{1/p} n - 1$ son $1/p$ veces la constante, $\log_{1/p} n - 2$ son $1/p^2$ veces la constante, etc.

A diferencia de los algoritmos anteriores, una *skip list* comienza vacía, y se va poblando insertando elementos a la lista. Se va colocando en la posición que no viole el orden; generando el nodo correspondiente con nivel calculado. Los nodos especiales *head* y *tail* siempre tienen el nivel máximo posible. La inserción de un valor encapsulado en el nodo u comienza por visitar el máximo nivel en *head* e ir bajando hasta determinar $u.dato > head[level].dato$; en ese momento se debe avanzar al nodo apuntado por $head[level]$ y repetir el algoritmo hasta que $level = 1$, en cuyo caso encontramos el lugar de inserción del nuevo dato. Se procede a reasignar los punteros de los sucesores y ajustar los punteros hacia los nodos sucesores a los niveles que tiene u .

Cada inserción tiene un costo $O(\log_{1/p} n)$, garantía probabilística; por lo que insertar n elementos tiene un costo:

$$\sum_{i=1}^n O(\log_{1/p} i) = O(\log_{1/p} \prod_{i=1}^n i) = O(\log_{1/p} n!) = O(n \log n);$$

usando la aproximación de Stirling.

A diferencia de la versión basada en arreglos, una *skip list* es capaz de aceptar nuevos elementos y mantener el orden de manera eficiente.

1.3.1. Ejercicios:

1. ¿Cuál es el costo en memoria de una *skip list*? 1.1 Investigue, implemente y pruebe una *skip list*.
2. Investigue, implemente y pruebe un árbol binario de búsqueda.

1.4. Lecturas

Las lecturas de este tema corresponden al capítulo 5 de (Knuth 1998), en específico 5.2 *Internal sorting*. También se recomienda leer y comprender la parte II de (Cormen et al. 2022), que corresponde a *Sorting and order statistics*, en particular Cap. 6 y 7, así como el Cap. 8.1. El artículo de Wikipedia https://en.wikipedia.org/wiki/Sorting_algorithm también puede ser consultado con la idea de encontrar una explicación rápida de los algoritmos.

En la práctica, pocos algoritmos son mejores que *quicksort*. En (Loeser 1974) se detalla una serie de experimentos donde se compara quicksort contra otros algoritmos relacionados; por lo que es una lectura recomendable.

La parte adaptable, esto es para algoritmos *oportunistas* que toman ventaja de instancias simples, está cubierta por el artículo (Estivill-Castro y Wood 1992). En especial, es muy necesario comprender las secciones 1.1 y 1.2, el resto del artículo debe ser leído aunque no invierta mucho tiempo en comprender las pruebas expuestas si no le son claras. En especial, en las secciones indicadas se establecen las medidas de desorden contra las cuales se mide la complejidad. En (Cook y Kim 1980) se realiza una comparación del desempeño de varios algoritmos para ordenamiento de listas casi ordenadas, esto es, en cierto sentido donde los algoritmos adaptables tienen sentido. Este artículo es anterior a (Estivill-Castro y Wood 1992) pero tiene experimentos que simplifican el entendimiento de los temas.

1.5. Material audio-visual sobre algoritmos de ordenamiento

1.6. Actividades

1.6.1. Actividad 1: Explotando la localidad y tamaños de instancia

Aunque algoritmos como *insertion sort* o *adaptive bubble sort* tienen costos asintóticos subóptimos ($O(n^2)$) para su peor caso, su simplicidad y localidad en los accesos de memoria los hace muy eficientes para instancias pequeñas o arreglos que se encuentran “casi ordenados”. Por el contrario, algoritmos recursivos óptimos como *quick sort* o *merge sort* tienen un *overhead* asociado por las llamadas a funciones y uso de memoria adicional que los penalizan con arreglos pequeños.

El objetivo de esta actividad empírica es determinar empíricamente los umbrales de tamaño donde los algoritmos adaptativos y simples superan a los algoritmos óptimos $O(n \log n)$.

1. Escriba un generador de datos para crear instancias aleatorias de tamaño variable $n \in \{4, 8, 16, 32, 64, 128, 256, 512\}$. **No** utilice datos pre-cargados ni archivos; debe generar los arreglos en su código.
2. Escriba un generador secundario para instancias “casi ordenadas”. Puede lograr esto generando un arreglo ordenado y aplicando un número de intercambios equivalente al 5 % o 10 % del tamaño n .
3. Mida y compare los tiempos reales de ejecución (usando herramientas como `@benchmark` en Julia) y el número de comparaciones de *insertion sort*, *adaptive bubble sort*, *quick sort*, *merge sort* y *heap sort* en todos los tamaños descritos.
4. Identifique y reporte mediante gráficas y tablas el “punto de cruce”, es decir, el tamaño n en el cual *quick sort* y *merge sort* comienzan a superar consistentemente a *insertion sort* para las instancias puramente aleatorias.
5. Reporte si el umbral cambia (y cuánto) para el caso de las instancias casi ordenadas.
6. Discuta los hallazgos en términos de complejidad asintótica y cómo las constantes y la localidad de la memoria actúan en la práctica.

1.6.2. Entregable

Su trabajo se entregará en PDF y con el notebook fuente, se entregará en el sistema en archivos separados. El código se debe documentar, así como mantener una estructura del documento que permita a un lector interesado entender el problema, sus experimentos y metodología, así como sus conclusiones. Tenga en cuenta que los notebooks pueden alternar celdas de texto y código.

No olvide estructurar su reporte, en particular el reporte debe cubrir los siguientes puntos:

- Título del reporte, su nombre.
- Introducción.
- Desarrollo de las actividades.
- Conclusiones
- Lista de referencias. Nota, una lista de referencias que no fueron utilizadas en el cuerpo del texto será interpretada como una lista vacía.

Nota: no olvides revisar la rúbrica del curso.

Cook, Curtis R, y Do Jin Kim. 1980. “Best sorting algorithm for nearly sorted lists”. *Communications of the ACM* 23 (11): 620–24.

Cormen, Thomas H, Charles E Leiserson, Ronald L Rivest, y Clifford Stein. 2022. *Introduction to algorithms*. MIT press.

Estivill-Castro, Vladmir, y Derick Wood. 1992. “A survey of adaptive sorting algorithms”. *ACM Computing Surveys (CSUR)* 24 (4): 441–76.

Knuth, Donald. 1998. *The Art Of Computer Programming, vol. 3 (2nd ed): Sorting And Searching*. Vol. 3. Addison Wesley Longman Publishing Co. Inc.

Loeser, Rudolf. 1974. “Some performance tests of ‘quicksort’ and descendants”. *Communications of the ACM* 17 (3): 143–52.

Pugh, William. 1990. “Skip lists: a probabilistic alternative to balanced trees”. *Commun. ACM* (New York, NY, USA) 33 (6): 668–76. <https://doi.org/10.1145/78973.78977>.

Listado 4 Algoritmo *heap sort*

```
function heapify!(A, n, i)
    largest = i
    l = 2i
    r = 2i + 1

    if l <= n && A[l] > A[largest]
        largest = l
    end
    if r <= n && A[r] > A[largest]
        largest = r
    end
    if largest != i
        A[i], A[largest] = A[largest], A[i]
        heapify!(A, n, largest)
    end
end

function heap_sort!(A)
    n = length(A)
    for i in n ÷ 2 : -1 : 1
        heapify!(A, n, i)
    end
    for i in n : -1 : 2
        A[1], A[i] = A[i], A[1]
        heapify!(A, i - 1, 1)
    end

    A
end

heap_sort!([6, 8, 3, 7, 4, 1, 2, 5])
```

Listado 5 Algoritmo *merge sort*

```
function merge!(A, low, mid, high, temp)
    for i in low:high
        temp[i] = A[i]
    end
    i = low
    j = mid + 1
    k = low
    while i <= mid && j <= high
        if temp[i] <= temp[j]
            A[k] = temp[i]
            i += 1
        else
            A[k] = temp[j]
            j += 1
        end
        k += 1
    end
    while i <= mid
        A[k] = temp[i]
        i += 1
        k += 1
    end
end

function merge_sort!(A, low=1, high=length(A), temp=similar(A))
    if low < high
        mid = low + (high - low) ÷ 2
        merge_sort!(A, low, mid, temp)
        merge_sort!(A, mid + 1, high, temp)
        merge!(A, low, mid, high, temp)
    end

    A
end

merge_sort!([6, 8, 3, 7, 4, 1, 2, 5])
```

Listado 6 Algoritmo *quick sort*.

```
using Random
```

```
function qsort!(A, low=1, high=length(A))  
    if low < high  
        piv = part!(A, low, high) ①  
        qsort!(A, low, piv - 1) ②  
        qsort!(A, piv + 1, high)  
    end
```

```
    A  
end
```

```
function part!(A, low, high)  
    ipiv = rand(low:high) ③  
    A[ipiv], A[high] = A[high], A[ipiv]  
    piv = A[high]
```

```
    i = low - 1 # uno antes porque se accede después de un i+1  
    for j in low:high - 1 ④  
        if A[j] < piv  
            i += 1  
            A[i], A[j] = A[j], A[i] ⑤  
        end  
    end
```

```
    ipiv = i + 1  
    A[ipiv], A[high] = A[high], A[ipiv] ⑥  
    ipiv  
end
```

```
qsort!([6, 8, 3, 7, 4, 1, 2, 5])
```
